Implement a Menu- Driven Doubly Linked List In C

```
#include <stdio.h>
#include <stdlib.h>

// Structure of a doubly linked list node
struct Node {
   int data;
   struct Node* next;
   struct Node* prev;
};

// Head pointer of the first list
struct Node* head = NULL;

// Function to display the list
void display() {
  if(head == NULL){
            printf("List is empty\n");
            return;
          }
          struct Node* temp = head;
          while(temp != NULL){
                    printf("%d ", temp -> data);
                    temp = temp -> next;
          }
          printf("\n");
}

// Function to insert a number at the end
void insert(int num) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
        newNode -> data = num;
        newNode -> next = NULL;
        newNode -> prev = NULL;
        if(head == NULL){
            head = newNode;
        } else{
            struct Node* temp = head;
            while(temp -> next != NULL)
                temp = temp -> next;
            temp -> next = newNode;
            newNode -> prev = temp;
        }
        printf("Inserted successfully\n");
}

// Function to delete a specific number
void deleteNumber(int num) {
    if(head == NULL){
            printf("List is empty\n");
            return;
        }
        struct Node* temp = head;
        while(temp != NULL && temp -> data != num)
            temp = temp -> next;
        if(temp == NULL){
            printf("Number %d not found\n", num);
            return;
        }
        if(temp == head){
            head = temp -> next;
```

```
		if(head != NULL)
			head -> prev = NULL;
	}else {
		temp -> prev -> next = temp -> next;
		if(temp -> next != NULL)
			temp -> next -> prev = temp -> prev;
	}
	free(temp);
	printf("List after deletion: ");
	display();
}

// Function to reverse the list
void reverseList() {
	if(head == NULL){
		printf("List is empty\n");
		return;
	}
	struct Node* temp = NULL;
	struct Node* current = head;
	while(current != NULL){
		temp = current -> prev;
		current -> prev = current -> next;
		current -> next = temp;
		current = current -> prev;
	}
	if(temp != NULL)
		head = temp -> prev;
	display();

}
```

```
// Function to concatenate another list
void concatenate() {
	int n, x;
	scanf("%d", &n);
	if(n == 0){
		printf("Second list is empty\n");
		return;
	}
	struct Node* second = NULL;
	struct Node* last = NULL;
	for(int i = 0; i < n; i++){
		scanf("%d", &x);
		struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
		newNode -> data = x;
		newNode -> next = NULL;
		newNode -> prev = NULL;
		if(second == NULL){
			second = newNode;
			last = newNode;
		}else{
			last -> next = newNode;
			newNode -> prev = last;
			last = newNode;
		}
	}
	if(head == NULL){
		head = second;
	}else{
		struct Node* temp = head;
		while(temp -> next != NULL)
```

```
                temp = temp -> next;
            temp -> next = second;
            second -> prev = temp;
        }
        display();
}

int main() {
    int choice, num;

    while (1) {
        printf("Enter operation\n1: Insert\n2: Delete\n3: Display\n4: Reverse\n5: Concatenate\n6: Exit\n");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                scanf("%d", &num);
                insert(num);
                break;

            case 2:
                scanf("%d", &num);
                deleteNumber(num);
                break;

            case 3:
                display();
                break;

            case 4:
```

```
            reverseList();
            break;

        case 5:
            concatenate();
            break;

        case 6:
            printf("Exit\n");
            exit(0);

        default:
            printf("Invalid choice\n");
      }
   }

   return 0;
}
```